Sistemi bazirani na znanju - Prijedlog projekta
Slaven Garić - sw47/2015

Ekspertski sistem za hidrološki monitoring

1. Motivacija

Upravljanje vodnim resursima predstavlja jedan od najvećih infrastrukturnih izazova u zemljama s niskim nadmorskim visinama. Holandija je posebno karakterističan primjer, oko 26% njene teritorije nalazi se ispod nivoa mora, a gotovo 60% stanovništva živi u zonama ugroženim poplavama. Razrađeni sistem kanala, nasipa, crpnih stanica (pumpnih stanica) i brana čini okosnicu zaštite, ali zahtijeva kontinuirani nadzor u realnom vremenu.

Savremeni SCADA (Supervisory Control and Data Acquisition) sistemi koji se koriste za hidrološki monitoring uglavnom se oslanjaju na statičke pragove za generisanje alarma. Kada nivo vode pređe fiksnu vrijednost, aktivira se alarm. Ovakav pristup, iako funkcionalan, ima značajna ograničenja: ne uzima u obzir širi kontekst situacije, poput intenziteta padavina, stanja uzvodnih stanica ili operativnog kapaciteta crpnih stanica.

Praktičan problem s kojim se susreću operateri jeste kognitivno opterećenje. U tipičnoj SCADA aplikaciji, operater mora da prati parametre i indikatore raspoređene kroz brojne različite komponente i stranice grafičkog interfejsa (grafove trendova, tabele alarma, mape lokacija, statuse pumpi, meteorološke podatke) i da samostalno u glavi kombinuje sve te informacije kako bi procjenio da li postoji stvarna opasnost. Kada se nadzire mreža od desetine ili stotine mjernih stanica, ovaj manualni proces postaje izuzetno naporan i podložan greškama, naročito tokom noćnih smjena ili dužih perioda intenzivnih padavina.

Cilj ovog projekta je razvoj ekspertskog sistema za hidrološki monitoring koji primjenom sistema baziranog na znanju nadograđuje tradicionalni SCADA pristup. Sistem će automatski obavljati višeslojno rezonovanje od klasifikacije sirovih senzorskih podataka, preko procjene situacije, do konkretnih preporuka za akciju čime se značajno smanjuje kognitivno opterećenje operatera i ubrzava proces donošenja odluka u kritičnim situacijama.

2. Pregled problema

Hidrološki monitoring obuhvata kontinuirano praćenje nivoa vode, protoka, rada crpnih stanica i meteoroloških uslova na mreži mjernih lokacija (kanali, akumulacije, crpne stanice). Operateri moraju na osnovu velikog broja senzorskih podataka donositi brže i tačne odluke o nivou opasnosti i potrebnim intervencijama.

2.1 Nedostaci postojećih rješenja:

Kod nekih od postojećih SCADA sistema iz ovog domena postoje ključne mane u njihovim rješenjima:

- **Statički pragovi alarma**

  Alarmi se aktiviraju isključivo na osnovu fiksnih vrijednosti jednog senzora, bez razmatranja konteksta. Ovo rezultuje velikim brojem lažnih alarma (npr. kratkotrajan

skok nivoa koji nije opasan) ili propuštenih kritičnih situacija (npr. umjereno povišen nivo, ali uz najavljene intenzivne padavine i smanjen kapacitet pumpi).

- **Nepostojanje višeslojnog rezonovanja**

  Sistem ne može da kombinuje podatke iz više izvora i izvede zaključke u više koraka. Na primjer, ne može automatski da zaključi: povišen nivo vode + intenzivne padavine + smanjen kapacitet crpne stanice = kritična situacija koja zahtijeva hitnu intervenciju.

- **Raspršenost informacija po aplikaciji**

  Operater mora manualno da kruži između različitih stranica i elemenata grafičkog interfejsa (trendovi, alarmi, mape, statusi) da bi stekao cjelovitu sliku. Ne postoji jedinstven mehanizam koji bi automatski sintetizovao sve relevantne informacije u jasnu procjenu rizika i preporuku za akciju.

- **Odsustvo detekcije obrazaca u realnom vremenu**

  Sistem ne prepoznaje brze promjene (npr. nagli porast vodostaja od 50cm za 30 minuta) niti korelacije između uzvodnih i nizvodnih stanica koje ukazuju na propagaciju poplavnog talasa.

- **Nema analize uzročno-posledičnih veza**

  Operater ne može automatski da utvrdi zašto je vodostaj na nekoj lokaciji kritičan, da li je uzrok uzvodna situacija, kvar crpne stanice ili intenzivne padavine.

2.2 Prednosti predloženog rješenja

Predloženi sistem rješava navedene nedostatke primjenom Drools rule engine-a kroz sledeće mehanizme:

- **Forward chaining rezonovanje**

  Višeslojno rezonovanje gdje prvi nivo pravila klasifikuje sirove senzorske podatke u kategorije stanja, zatim drugi nivo kombinuje ta stanja sa meteorološkim podacima u procjenu rizika i treći nivo generiše konkretne preporuke za intervenciju i sistemska upozorenja.

- **Complex Event Processing (CEP)**

  Detekcija kritičnih obrazaca u tokovima senzorskih događaja: nagli porast vodostaja, višestruki kvarovi pumpi, gubitak komunikacije sa stanicom, i propagacija poplavnog talasa duž hidrografske mreže.

- **Rule templates**

  Parametrizovana pravila za klasifikaciju vodostaja prema tipu lokacije (rijeka, kanal, akumulacija, crpna stanica), čime se izbjegava dupliranje pravila i omogućava lako dodavanje novih tipova.

3. Metodologija rada

3.1 Arhitektura sistema

Sistem se realizuje kao web aplikacija sa sledećim komponentama: React frontend za prikaz dashboarda i interakciju sa operaterima, Spring Boot backend sa REST API-jem i integrisanim Drools rule engine-om i MySQL baza podataka za skladištenje entiteta i istorijskih podataka.

3.2 Korisničke uloge

Administrator

Upravljanje korisnicima, lokacijama, senzorima i konfiguracijama pragova. Zadužen je za CRUD operacije nad svim entitetima sistema.

Operater

Pregled dashboarda sa trenutnim stanjem sistema, pokretanje rezonera za procjenu rizika, pregled i prihvatanje alarma, pregled notifikacija dobijenih na osnovu CEP događaja.

3.3 Ulazi u sistem (Input)

Senzorski podaci u realnom vremenu

Očitavanja sa mjernih instrumenata: nivo vode (cm), protok vode (m^3/s), status pumpe (aktivna/neaktivna/u kvaru/na održavanju). Svako očitavanje sadrži: lokacijski kod, naziv senzora, vremenski pečat, vrijednost i indikator validnosti.

Meteorološki podaci

Količina padavina (mm/h), prognoza padavina za narednih 24h (mm).

Konfiguracioni podaci

Definicije lokacija sa tipom (RIVER/CANAL/RESERVOIR/PUMP_STATION), definicije senzora sa pragovima, konfiguracije pragova po kombinaciji tipa lokacije i parametra (za Grupu 1 pravila i templejte).

Korisničke akcije

Zahtjevi operatera za pokretanje rezonera, prihvatanje/resetovanje alarma, simulacija senzorskih očitavanja za testiranje.

3.4 Izlazi iz sistema (Output)

Klasifikacija stanja (prvi nivo)

Za svaku lokaciju: status nivoa vode (NORMAL / ELEVATED / HIGH / CRITICAL), status protoka (LOW / NORMAL / HIGH), operativni status svake pumpe (ACTIVE / IDLE / FAULTY / MAINTENANCE).

Procjena rizika i kapaciteta (drugi nivo)

Za svaku lokaciju: nivo rizika od poplave (LOW / MODERATE / HIGH / EXTREME) sa obrazloženjem faktora. Za crpne stanice: operativni kapacitet (FULL / REDUCED / MINIMAL / OFFLINE).

Preporuke i upozorenja (treći nivo)

Konkretne preporuke za intervenciju po lokaciji (MONITOR / PREPARE / ACTIVATE / EVACUATE). Globalni nivo uzbune sistema (GREEN / YELLOW / ORANGE / RED).

CEP notifikacije

Detekcija naglog porasta vodostaja, obrasca kvara pumpe, gubitka komunikacije sa stanicom i gubitka komunikacije sa pojedinačnom pumpom.

3.5 Baza znanja - Kompletan pregled pravila

Grupa 1: Forward chaining Klasifikacija senzorskih podataka

Ova grupa pravila prima sirova senzorska očitavanja i klasifikuje ih u kategorije stanja. Pragovi za nivo vode i protok su definisani od strane admina.

Pravilo 1.1 - Klasifikacija nivoa vode: NORMAL

```
rule "Klasifikacija nivoa vode - NORMAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              normalMax >= $val)
    not WaterLevelStatus(location == $loc)
  then
    insert(new WaterLevelStatus($loc, StatusLevel.NORMAL, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.2 - Klasifikacija nivoa vode: ELEVATED

```
rule "Klasifikacija nivoa vode - ELEVATED"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              normalMax < $val, warningMax >= $val)
    not WaterLevelStatus(location == $loc)
  then
    insert(new WaterLevelStatus($loc, StatusLevel.ELEVATED, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.3 - Klasifikacija nivoa vode: HIGH

```
rule "Klasifikacija nivoa vode - HIGH"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
```

```
            warningMax < $val, criticalMax >= $val)
    not WaterLevelStatus(location == $loc)
  then
    insert(new WaterLevelStatus($loc, StatusLevel.HIGH, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.4 - Klasifikacija nivoa vode: CRITICAL

```
rule "Klasifikacija nivoa vode - CRITICAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              criticalMax < $val)
    not WaterLevelStatus(location == $loc)
  then
    insert(new WaterLevelStatus($loc, StatusLevel.CRITICAL, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.5 - Ažuriranje nivoa vode: NORMAL

```
rule "Ažuriranje nivoa vode - NORMAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              normalMax >= $val)
    $existing: WaterLevelStatus(location == $loc, level != StatusLevel.NORMAL,
timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(StatusLevel.NORMAL), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.6 - Ažuriranje nivoa vode: ELEVATED

```
rule "Ažuriranje nivoa vode - ELEVATED"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              normalMax < $val, warningMax >= $val)
    $existing: WaterLevelStatus(location == $loc, level !=
StatusLevel.ELEVATED, timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(StatusLevel.ELEVATED), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.7 - Ažuriranje nivoa vode: HIGH

```
rule "Ažuriranje nivoa vode - HIGH"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              warningMax < $val, criticalMax >= $val)
    $existing: WaterLevelStatus(location == $loc, level != StatusLevel.HIGH,
timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(StatusLevel.HIGH), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.8 - Ažuriranje nivoa vode: CRITICAL

```
rule "Ažuriranje nivoa vode - CRITICAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.WATER_LEVEL,
              criticalMax < $val)
    $existing: WaterLevelStatus(location == $loc, level !=
StatusLevel.CRITICAL, timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(StatusLevel.CRITICAL), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.9 - Klasifikacija protoka: LOW

```
rule "Klasifikacija protoka - LOW"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.FLOW_RATE,
              normalMax > $val)
    not FlowRateStatus(location == $loc)
  then
    insert(new FlowRateStatus($loc, FlowLevel.LOW, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.10 - Klasifikacija protoka: NORMAL

```
rule "Klasifikacija protoka - NORMAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
```

```
            parameterType == ParameterType.FLOW_RATE,
            normalMax <= $val, warningMax >= $val)
    not FlowRateStatus(location == $loc)
  then
    insert(new FlowRateStatus($loc, FlowLevel.NORMAL, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.11 - Klasifikacija protoka: HIGH

```
rule "Klasifikacija protoka - HIGH"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
            $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
            parameterType == ParameterType.FLOW_RATE,
            warningMax < $val)
    not FlowRateStatus(location == $loc)
  then
    insert(new FlowRateStatus($loc, FlowLevel.HIGH, $val,
$reading.getTimestamp()));
end
```

Pravilo 1.12 - Ažuriranje protoka: LOW

```
rule "Ažuriranje protoka - LOW"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
            $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
            parameterType == ParameterType.FLOW_RATE,
            normalMax > $val)
    $existing: FlowRateStatus(location == $loc, level != FlowLevel.LOW,
timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(FlowLevel.LOW), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.13 - Ažuriranje protoka: NORMAL

```
rule "Ažuriranje protoka - NORMAL"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
            $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
            parameterType == ParameterType.FLOW_RATE,
            normalMax <= $val, warningMax >= $val)
    $existing: FlowRateStatus(location == $loc, level != FlowLevel.NORMAL,
timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(FlowLevel.NORMAL), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.14 - Ažuriranje protoka: HIGH

```
rule "Ažuriranje protoka - HIGH"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.FLOW_RATE,
              $loc: location, $val: value)
    $thresh: ThresholdConfig(locationType == $loc.type,
              parameterType == ParameterType.FLOW_RATE,
              warningMax < $val)
    $existing: FlowRateStatus(location == $loc, level != FlowLevel.HIGH,
timestamp < $reading.timestamp)
  then
    modify($existing) { setLevel(FlowLevel.HIGH), setValue($val),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.15 - Status pumpe: ACTIVE

```
rule "Status pumpe - ACTIVE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
              $loc: location, $pumpId: sensorName, value == 1.0)
    not PumpOperationalStatus(location == $loc, pumpId == $pumpId)
  then
    insert(new PumpOperationalStatus($loc, $pumpId, PumpState.ACTIVE,
$reading.getTimestamp()));
end
```

Pravilo 1.16 - Status pumpe: IDLE

```
rule "Status pumpe - IDLE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
              $loc: location, $pumpId: sensorName, value == 0.0)
    not PumpOperationalStatus(location == $loc, pumpId == $pumpId)
  then
    insert(new PumpOperationalStatus($loc, $pumpId, PumpState.IDLE,
$reading.getTimestamp()));
end
```

Pravilo 1.17 - Status pumpe: FAULTY

```
rule "Status pumpe - FAULTY"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
              $loc: location, $pumpId: sensorName, value == -1.0)
    not PumpOperationalStatus(location == $loc, pumpId == $pumpId)
  then
    insert(new PumpOperationalStatus($loc, $pumpId, PumpState.FAULTY,
$reading.getTimestamp()));
end
```

Pravilo 1.18 - Status pumpe: MAINTENANCE

```
rule "Status pumpe - MAINTENANCE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
             $loc: location, $pumpId: sensorName, value == -2.0)
    not PumpOperationalStatus(location == $loc, pumpId == $pumpId)
  then
    insert(new PumpOperationalStatus($loc, $pumpId, PumpState.MAINTENANCE,
$reading.getTimestamp()));
end
```

Pravilo 1.19 - Ažuriranje statusa pumpe: ACTIVE

```
rule "Ažuriranje statusa pumpe - ACTIVE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
             $loc: location, $pumpId: sensorName, value == 1.0)
    $existing: PumpOperationalStatus(location == $loc,
              pumpId == $pumpId, state != PumpState.ACTIVE, timestamp <
$reading.timestamp)
  then
    modify($existing) { setState(PumpState.ACTIVE),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.20 - Ažuriranje statusa pumpe: IDLE

```
rule "Ažuriranje statusa pumpe - IDLE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
             $loc: location, $pumpId: sensorName, value == 0.0)
    $existing: PumpOperationalStatus(location == $loc,
              pumpId == $pumpId, state != PumpState.IDLE, timestamp <
$reading.timestamp)
  then
     modify($existing) { setState(PumpState.IDLE),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.21 - Ažuriranje statusa pumpe: FAULTY

```
rule "Ažuriranje statusa pumpe - FAULTY"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
             $loc: location, $pumpId: sensorName, value == -1.0)
    $existing: PumpOperationalStatus(location == $loc,
              pumpId == $pumpId, state != PumpState.FAULTY, timestamp <
$reading.timestamp)
  then
    modify($existing) { setState(PumpState.FAULTY),
setTimestamp($reading.getTimestamp()) };
end
```

Pravilo 1.22 - Ažuriranje statusa pumpe: MAINTENANCE

```
rule "Ažuriranje statusa pumpe - MAINTENANCE"
  salience 100
  when
    $reading: SensorReading(sensorType == SensorType.PUMP_STATUS,
             $loc: location, $pumpId: sensorName, value == -2.0)
    $existing: PumpOperationalStatus(location == $loc,
              pumpId == $pumpId, state != PumpState.MAINTENANCE, timestamp <
$reading.timestamp)
  then
    modify($existing) { setState(PumpState.MAINTENANCE),
setTimestamp($reading.getTimestamp()) };
end
```

Grupa 2: Forward chaining - Procjena situacije

Ova grupa pravila kombinuje klasifikovane statuse iz prvog nivoa sa meteorološkim podacima.

Pravilo 2.1 - Rizik od poplave: LOW

```
rule "Rizik od poplave - LOW"
  salience 50
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.NORMAL)
    FlowRateStatus(location == $loc, level == FlowLevel.NORMAL)
    WeatherCondition(location == $loc, precipitation < 10)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.LOW,
      "Nivo i protok normalni, padavine minimalne"));
end
```

Pravilo 2.2 - Rizik od poplave: MODERATE (povišen nivo)

```
rule "Rizik - MODERATE - povišen nivo"
  salience 60
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.ELEVATED)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.MODERATE,
      "Povišen nivo vode"));
end
```

Pravilo 2.3 - Rizik od poplave: MODERATE (normalan nivo + jake padavine)

```
rule "Rizik - MODERATE - padavine"
  salience 60
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.NORMAL)
    WeatherCondition(location == $loc, precipitation > 30)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.MODERATE,
      "Nivo normalan, ali intenzivne padavine"));
```

```
end
```

Pravilo 2.4 - Rizik od poplave: HIGH (visok nivo)

```
rule "Rizik - HIGH - visok nivo"
  salience 70
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.HIGH)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.HIGH,
      "Visok nivo vode"));
end
```

Pravilo 2.5 - Rizik od poplave: HIGH (povišen nivo + jake padavine)

```
rule "Rizik - HIGH - povisen nivo i padavine"
  salience 70
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.ELEVATED)
    WeatherCondition(location == $loc, precipitation > 50)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.HIGH,
      "Povišen nivo i veoma intenzivne padavine"));
end
```

Pravilo 2.6 - Rizik od poplave: EXTREME (kritičan nivo)

```
rule "Rizik - EXTREME - kritican nivo"
  salience 80
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.CRITICAL)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.EXTREME,
      "Kritičan nivo vode"));
end
```

Pravilo 2.7 - Rizik od poplave: EXTREME (visok nivo + visok protok)

```
rule "Rizik - EXTREME - visok nivo i protok"
  salience 80
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.HIGH)
    FlowRateStatus(location == $loc, level == FlowLevel.HIGH)
    not FloodRiskAssessment(location == $loc)
  then
    insert(new FloodRiskAssessment($loc, RiskLevel.EXTREME,
      "Visok nivo i visok protok istovremeno"));
end
```

Pravilo 2.8 - Ažuriranje rizika: LOW => MODERATE - povišen nivo

```
rule "Ažuriranje rizika - MODERATE - povišen nivo"
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.ELEVATED)
    $risk: FloodRiskAssessment(location == $loc,
           riskLevel == RiskLevel.LOW)
  then
    modify($risk) { setRiskLevel(RiskLevel.MODERATE),
      setReason("Nivo vode porastao na ELEVATED")
    };
end
```

Pravilo 2.9 - Ažuriranje rizika: eskalacija na HIGH

```
rule "Ažuriranje rizika - eskalacija na HIGH"
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.HIGH)
    $risk: FloodRiskAssessment(location == $loc,
           riskLevel != RiskLevel.HIGH, riskLevel != RiskLevel.EXTREME)
  then
    modify($risk) { setRiskLevel(RiskLevel.HIGH),
      setReason("Nivo vode porastao na HIGH")
    };
end
```

Pravilo 2.10 - Ažuriranje rizika: eskalacija na EXTREME

```
rule "Ažuriranje rizika - eskalacija na EXTREME"
  when
    $loc: Location()
    WaterLevelStatus(location == $loc, level == StatusLevel.CRITICAL)
    $risk: FloodRiskAssessment(location == $loc,
           riskLevel != RiskLevel.EXTREME)
  then
    modify($risk) { setRiskLevel(RiskLevel.EXTREME),
      setReason("Nivo vode dostigao CRITICAL")
    };
end
```

Pravilo 2.11 - Kapacitet crpne stanice: FULL (≥80% aktivnih)

```
rule "Kapacitet stanice - FULL (>=80% aktivnih)"
  salience 50
  when
    $loc: Location(type == LocationType.PUMP_STATION)
    Number($t: intValue, intValue > 0) from accumulate(
      PumpOperationalStatus(location == $loc), count(1))
    Number($a: intValue) from accumulate(
      PumpOperationalStatus(location == $loc,
        state == PumpState.ACTIVE), count(1))
    eval((double) $a / $t >= 0.8)
    not StationCapacity(location == $loc)
  then
```

```
    insert(new StationCapacity($loc, CapacityLevel.FULL,
      $a, $t));
end
```

Pravilo 2.12 - Kapacitet crpne stanice: REDUCED (50–80% aktivnih)

```
rule "Kapacitet stanice - REDUCED (50-80% aktivnih)"
  salience 50
  when
    $loc: Location(type == LocationType.PUMP_STATION)
    Number($t: intValue, intValue > 0) from accumulate(
      PumpOperationalStatus(location == $loc), count(1))
    Number($a: intValue) from accumulate(
      PumpOperationalStatus(location == $loc,
        state == PumpState.ACTIVE), count(1))
    eval((double) $a / $t >= 0.5)
    eval((double) $a / $t < 0.8)
    not StationCapacity(location == $loc)
  then
    insert(new StationCapacity($loc, CapacityLevel.REDUCED,
      $a, $t));
end
```

Pravilo 2.13 - Kapacitet crpne stanice: MINIMAL (20–50% aktivnih)

```
rule "Kapacitet stanice - MINIMAL (20-50% aktivnih)"
  salience 50
  when
    $loc: Location(type == LocationType.PUMP_STATION)
    Number($t: intValue, intValue > 0) from accumulate(
      PumpOperationalStatus(location == $loc), count(1))
    Number($a: intValue) from accumulate(
      PumpOperationalStatus(location == $loc,
        state == PumpState.ACTIVE), count(1))
    eval((double) $a / $t >= 0.2)
    eval((double) $a / $t < 0.5)
    not StationCapacity(location == $loc)
  then
    insert(new StationCapacity($loc, CapacityLevel.MINIMAL,
      $a, $t));
end
```

Pravilo 2.14 - Kapacitet crpne stanice: OFFLINE (<20% aktivnih)

```
rule "Kapacitet stanice - OFFLINE (<20% aktivnih)"
  salience 50
  when
    $loc: Location(type == LocationType.PUMP_STATION)
    Number($t: intValue, intValue > 0) from accumulate(
      PumpOperationalStatus(location == $loc), count(1))
    Number($a: intValue) from accumulate(
      PumpOperationalStatus(location == $loc,
        state == PumpState.ACTIVE), count(1))
    eval((double) $a / $t < 0.2)
    not StationCapacity(location == $loc)
  then
    insert(new StationCapacity($loc, CapacityLevel.OFFLINE,
```

```
      $a, $t));
end
```

Pravilo 2.15 - Ažuriranje kapaciteta stanice - promjena aktivnih pumpi na FULL

```
rule "Azuriranje kapaciteta stanice - FULL - promjena aktivnih pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.8)
        $existing: StationCapacity(location == $loc, activePumps != $a)
    then
        modify($existing) {
            setLevel(CapacityLevel.FULL),
            setActivePumps($a)
        };
end
```

Pravilo 2.16 - Ažuriranje kapaciteta stanice - promjena ukupnog broja pumpi na FULL

```
rule "Azuriranje kapaciteta stanice - FULL - promjena ukupnog broja pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.8)
        $existing: StationCapacity(location == $loc, totalPumps != $t)
    then
        modify($existing) {
            setLevel(CapacityLevel.FULL),
            setTotalPumps($t)
        };
End
```

Pravilo 2.17 - Ažuriranje kapaciteta stanice - promjena aktivnih pumpi na REDUCED

```
rule "Azuriranje kapaciteta stanice - REDUCED - promjena aktivnih pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.5)
        eval((double) $a / $t < 0.8)
        $existing: StationCapacity(location == $loc, activePumps != $a)
    then
```

```
        modify($existing) {
            setLevel(CapacityLevel.REDUCED),
            setActivePumps($a)
        };
End
```

Pravilo 2.18 - Ažuriranje kapaciteta stanice - promjena ukupnog broja pumpi na REDUCED

```
rule "Azuriranje kapaciteta stanice - REDUCED - promjena ukupnog broja pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.5)
        eval((double) $a / $t < 0.8)
        $existing: StationCapacity(location == $loc, totalPumps != $t)
    then
        modify($existing) {
            setLevel(CapacityLevel.REDUCED),
            setTotalPumps($t)
        };
End
```

Pravilo 2.19 - Ažuriranje kapaciteta stanice - promjena aktivnih pumpi na MINIMAL

```
rule "Azuriranje kapaciteta stanice - MINIMAL - promjena aktivnih pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.2)
        eval((double) $a / $t < 0.5)
        $existing: StationCapacity(location == $loc, activePumps != $a)
    then
        modify($existing) {
            setLevel(CapacityLevel.MINIMAL),
            setActivePumps($a)
        };
End
```

Pravilo 2.20 - Ažuriranje kapaciteta stanice - promjena ukupnog broja pumpi na MINIMAL

```
rule "Azuriranje kapaciteta stanice - MINIMAL - promjena ukupnog broja pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
```

```
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t >= 0.2)
        eval((double) $a / $t < 0.5)
        $existing: StationCapacity(location == $loc, totalPumps != $t)
    then
        modify($existing) {
            setLevel(CapacityLevel.MINIMAL),
            setTotalPumps($t)
        };
End
```

Pravilo 2.21 - Ažuriranje kapaciteta stanice - promjena aktivnih pumpi na OFFLINE

```
rule "Azuriranje kapaciteta stanice - OFFLINE - promjena aktivnih pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t < 0.2)
        $existing: StationCapacity(location == $loc, activePumps != $a)
    then
        modify($existing) {
            setLevel(CapacityLevel.OFFLINE),
            setActivePumps($a)
        };
End
```

Pravilo 2.22 - Ažuriranje kapaciteta stanice - promjena ukupnog broja pumpi na OFFLINE

```
rule "Azuriranje kapaciteta stanice - OFFLINE - promjena ukupnog broja pumpi"
    salience 50
    when
        $loc: Location(type == LocationType.PUMP_STATION)
        Number($t: intValue, intValue > 0) from accumulate(
            PumpOperationalStatus(location == $loc), count(1))
        Number($a: intValue) from accumulate(
            PumpOperationalStatus(location == $loc, state == PumpState.ACTIVE),
            count(1))
        eval((double) $a / $t < 0.2)
        $existing: StationCapacity(location == $loc, totalPumps != $t)
    then
        modify($existing) {
            setLevel(CapacityLevel.OFFLINE),
            setTotalPumps($t)
        };
End
```

Pravilo 2.23 – Deeskalacija rizika – na HIGH

```
rule "Deeskalacija rizika - na HIGH"
```

```
    when
        $loc: Location()
        WaterLevelStatus(location == $loc, level == StatusLevel.HIGH)
        not FlowRateStatus(location == $loc, level == FlowLevel.HIGH)
        $risk: FloodRiskAssessment(location == $loc, riskLevel ==
RiskLevel.EXTREME)
    then
        modify($risk) {
            setRiskLevel(RiskLevel.HIGH),
            setReason("Nivo vode opao na HIGH")
        };
End
```

Pravilo 2.24 – Deeskalacija rizika – na MODERATE

```
rule "Deeskalacija rizika - na MODERATE"
    when
        $loc: Location()
        WaterLevelStatus(location == $loc, level == StatusLevel.ELEVATED)
        not WeatherCondition(location == $loc, precipitation > 50)
        $risk: FloodRiskAssessment(location == $loc,
                                   riskLevel != RiskLevel.MODERATE,
                                   riskLevel != RiskLevel.LOW)
    then
        modify($risk) {
            setRiskLevel(RiskLevel.MODERATE),
            setReason("Nivo vode opao na ELEVATED")
        };
End
```

Pravilo 2.25 – Deeskalacija rizika – na LOW

```
rule "Deeskalacija rizika - na LOW"
    when
        $loc: Location()
        WaterLevelStatus(location == $loc, level == StatusLevel.NORMAL)
        FlowRateStatus(location == $loc, level == FlowLevel.NORMAL)
        not WeatherCondition(location == $loc, precipitation >= 10)
        $risk: FloodRiskAssessment(location == $loc, riskLevel !=
RiskLevel.LOW)
    then
        modify($risk) {
            setRiskLevel(RiskLevel.LOW),
            setReason("Nivo i protok normalni, padavine minimalne")
        };
End
```

Grupa 3: Forward chaining - Preporuke za akciju

Treći nivo kombinuje procjene rizika iz drugog nivoa sa operativnim kapacitetom stanica i generiše preporuke za intervenciju i sistemska upozorenja.

Pravilo 3.1 - Preporuka: MONITOR (umjeren rizik, pun kapacitet)

```
rule "Preporuka - MONITOR - umjeren rizik"
  salience 10
```

```
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.MODERATE)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.MONITOR,
      Priority.MEDIUM, "Pojačan nadzor - umjeren rizik od poplave"));
end
```

Pravilo 3.2 - Preporuka: MONITOR (nizak rizik, smanjen kapacitet)

```
rule "Preporuka - MONITOR - smanjen kapacitet"
  salience 10
  when
    $loc: Location(type == LocationType.PUMP_STATION)
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.LOW)
    StationCapacity(location == $loc, level == CapacityLevel.REDUCED)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.MONITOR,
      Priority.MEDIUM, "Nadzor - nizak rizik ali smanjen kapacitet pumpi"));
end
```

Pravilo 3.3 - Preporuka: PREPARE (visok rizik, pun kapacitet)

```
rule "Preporuka - PREPARE - visok rizik pun kapacitet"
  salience 20
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.HIGH)
    StationCapacity(location == $loc, level == CapacityLevel.FULL)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.PREPARE,
      Priority.HIGH, "Pripremiti resurse - visok rizik, kapacitet pun"));
end
```

Pravilo 3.4 - Preporuka: PREPARE (visok rizik, smanjen kapacitet)

```
rule "Preporuka - PREPARE - visok rizik smanjen kapacitet"
  salience 20
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.HIGH)
    StationCapacity(location == $loc, level == CapacityLevel.REDUCED)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.PREPARE,
      Priority.HIGH,
      "Pripremiti resurse - visok rizik i smanjen kapacitet"));
End
```

Pravilo 3.5 - Generička preporuka PREPARE (visok rizik bez obzira na tip lokacije)

```
rule "Preporuka - PREPARE - generička za visok rizik"
  salience 20
  when
```

```
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.HIGH)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.PREPARE,
      Priority.HIGH, "Pripremiti resurse - visok rizik"));
end
```

Pravilo 3.6 - Preporuka: ACTIVATE (ekstreman rizik)

```
rule "Preporuka - ACTIVATE - ekstreman rizik"
  salience 30
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.EXTREME)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.ACTIVATE,
      Priority.CRITICAL,
      "Aktivirati vanredni režim - ekstreman rizik"));
end
```

Pravilo 3.7 - Preporuka: ACTIVATE (visok rizik + minimalan kapacitet)

```
rule "Preporuka - ACTIVATE - visok rizik i minimalan kapacitet"
  salience 30
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.HIGH)
    StationCapacity(location == $loc, level == CapacityLevel.MINIMAL)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.ACTIVATE,
      Priority.CRITICAL,
      "Aktivirati vanredni režim - visok rizik i minimalan kapacitet"));
end
```

Pravilo 3.8 - Preporuka: EVACUATE (ekstreman rizik + minimalan kapacitet)

```
rule "Preporuka - EVACUATE – ekstreman rizik i minimalan kapacitet"
  salience 40
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.EXTREME)
    StationCapacity(location == $loc, level == CapacityLevel.MINIMAL)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.EVACUATE,
      Priority.CRITICAL,
      "EVAKUACIJA - ekstreman rizik i minimalan kapacitet"));
end
```

Pravilo 3.9 - Preporuka: EVACUATE (ekstreman rizik + stanica offline)

```
rule "Preporuka - EVACUATE – ekstreman rizik i stanica offline"
  salience 40
```

```
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.EXTREME)
    StationCapacity(location == $loc, level == CapacityLevel.OFFLINE)
    not InterventionRecommendation(location == $loc)
  then
    insert(new InterventionRecommendation($loc, ActionType.EVACUATE,
      Priority.CRITICAL,
      "EVAKUACIJA - ekstreman rizik i stanica van funkcije"));
end
```

Pravilo 3.10 - Eskalacija preporuke MONITOR na PREPARE

```
rule "Eskalacija preporuke - MONITOR na PREPARE"
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.HIGH)
    $rec: InterventionRecommendation(location == $loc,
          type == ActionType.MONITOR)
  then
    modify($rec) { setType(ActionType.PREPARE),
      setPriority(Priority.HIGH),
      setDescription("Eskalacija - rizik porastao na HIGH")
    };
end
```

Pravilo 3.11 - Eskalacija preporuke na ACTIVATE

```
rule "Eskalacija preporuke - na ACTIVATE"
  when
    $loc: Location()
    FloodRiskAssessment(location == $loc, riskLevel == RiskLevel.EXTREME)
    $rec: InterventionRecommendation(location == $loc,
          type != ActionType.ACTIVATE, type != ActionType.EVACUATE)
  then
    modify($rec) { setType(ActionType.ACTIVATE),
      setPriority(Priority.CRITICAL),
      setDescription("Eskalacija - rizik dostigao EXTREME")
    };
end
```

Pravilo 3.12 - Sistemsko upozorenje - GREEN

```
rule "Sistemsko upozorenje - GREEN"
  salience -10
  when
    not FloodRiskAssessment(riskLevel == RiskLevel.HIGH)
    not FloodRiskAssessment(riskLevel == RiskLevel.EXTREME)
    Number(intValue == 0) from accumulate(
      InterventionRecommendation(type == ActionType.PREPARE),
      count(1))
    not SystemAlert()
  then
    insert(new SystemAlert(AlertLevel.GREEN,
      "Sistem u normalnom stanju"));
end
```

Pravilo 3.13 - Sistemsko upozorenje - YELLOW

```
rule "Sistemsko upozorenje - YELLOW"
  salience -10
  when
    Number(intValue >= 1) from accumulate(
      FloodRiskAssessment(riskLevel == RiskLevel.HIGH),
      count(1))
    Number(intValue == 0) from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE),
      count(1))
    not SystemAlert()
  then
    insert(new SystemAlert(AlertLevel.YELLOW,
      "ŽUTI ALARM - jedna ili više lokacija ima visok rizik"));
end
```

Pravilo 3.14 - Sistemsko upozorenje - ORANGE

```
rule "Sistemsko upozorenje - ORANGE"
  salience -10
  when
    $activateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE), count(1))
    $evacuateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.EVACUATE), count(1))
    eval(($activateCount.intValue() + $evacuateCount.intValue()) >= 1
      && ($activateCount.intValue() + $evacuateCount.intValue()) < 3)
    not SystemAlert()
  then
    int $count = $activateCount.intValue() + $evacuateCount.intValue();
    insert(new SystemAlert(AlertLevel.ORANGE,
      "NARANDZASTI ALARM - aktiviran vanredni rezim na " + $count + "
   lokacija(e)"));
end
```

Pravilo 3.15 - Sistemsko upozorenje - RED

```
rule "Sistemsko upozorenje - RED"
  salience -10
  when
    $activateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE), count(1))
    $evacuateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.EVACUATE), count(1))
    eval(($activateCount.intValue() + $evacuateCount.intValue()) >= 3)
    not SystemAlert()
  then
     insert(new SystemAlert(AlertLevel.RED, "CRVENI ALARM - vise lokacija u
       vanrednom rezimu"));
end
```

Pravilo 3.16 - Eskalacija sistemskog upozorenja na YELLOW

```
rule "Eskalacija sistemskog upozorenja - YELLOW"
  salience -10
  when
```

```
    Number(intValue >= 1) from accumulate(
      FloodRiskAssessment(riskLevel == RiskLevel.HIGH),
      count(1))
    Number(intValue == 0) from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE),
      count(1))
    $alert: SystemAlert(level == AlertLevel.GREEN)
  then
    modify($alert) { setLevel(AlertLevel.YELLOW),
      setDescription("ŽUTI ALARM - jedna ili više lokacija ima visok rizik")
    };
end
```

Pravilo 3.17 - Eskalacija sistemskog upozorenja na ORANGE iz GREEN

```
rule "Eskalacija sistemskog upozorenja - ORANGE iz GREEN"
  salience -10
  when
    $activateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE), count(1))
    $evacuateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.EVACUATE), count(1))
     eval(($activateCount.intValue() + $evacuateCount.intValue()) >= 1
       && ($activateCount.intValue() + $evacuateCount.intValue()) < 3)
     $alert: SystemAlert(level == AlertLevel.GREEN)
   then
     int $count = $activateCount.intValue() + $evacuateCount.intValue();
     modify($alert) {
       setLevel(AlertLevel.ORANGE),
       setDescription("NARANDZASTI ALARM - aktiviran vanredni rezim na "
        + $count + " lokacija(e)")
     };
end
```

Pravilo 3.18 - Eskalacija sistemskog upozorenja na ORANGE iz YELLOW

```
rule "Eskalacija sistemskog upozorenja – ORANGE iz YELLOW"
  salience -10
  when
    $activateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE), count(1))
    $evacuateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.EVACUATE), count(1))
        eval(($activateCount.intValue() + $evacuateCount.intValue()) >= 1
          && ($activateCount.intValue() + $evacuateCount.intValue()) < 3)
    $alert: SystemAlert(level == AlertLevel.YELLOW)
  then
    int $count = $activateCount.intValue() + $evacuateCount.intValue();
    modify($alert) {
      setLevel(AlertLevel.ORANGE),
      setDescription("NARANDZASTI ALARM - aktiviran vanredni rezim na "
                + $count + " lokacija(e)")
    };
end
```

Pravilo 3.19 - Eskalacija sistemskog upozorenja - RED

```
rule "Eskalacija sistemskog upozorenja - RED"
  salience -10
  when
    $activateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.ACTIVATE), count(1))
    $evacuateCount: Number() from accumulate(
      InterventionRecommendation(type == ActionType.EVACUATE), count(1))
      eval(($activateCount.intValue() + $evacuateCount.intValue()) >= 3)
      $alert: SystemAlert(level != AlertLevel.RED)
    then
        int $cnt = $activateCount.intValue() + $evacuateCount.intValue();
        modify($alert) {
            setLevel(AlertLevel.RED),
           setDescription("CRVENI ALARM - " + $cnt + " lokacija u vanrednom
        rezimu")
        };
end
```

Grupa 4: Complex Event Processing (CEP) - Monitoring u realnom vremenu

Pravilo 4.1 - Detekcija naglog porasta vodostaja

```
rule "CEP - Nagli porast vodostaja"
  when
    $r1: SensorReadingEvent(sensorType == SensorType.WATER_LEVEL,
         $loc: locationCode, $v1: value)
    $r2: SensorReadingEvent(sensorType == SensorType.WATER_LEVEL,
         locationCode == $loc,
         this after[0s, 30m] $r1,
         value >= $v1 + 50)
    not RapidWaterLevelRise(locationCode == $loc)
  then
    insert(new RapidWaterLevelRise($loc,
      $r2.getValue() - $v1, 30));
end
```

Pravilo 4.2 - Detekcija obrasca kvara pumpe

```
rule "CEP - Obrazac kvara pumpe"
  when
    $p1: PumpEvent(eventType == PumpEventType.RESTART,
         $loc: locationCode, $pumpId: pumpId)
    Number(intValue >= 3) from accumulate(
      PumpEvent(eventType == PumpEventType.RESTART,
              pumpId == $pumpId,
              this after[0s, 1h] $p1),
      count(1))
    not PumpFailureAlert(pumpId == $pumpId)
  then
    insert(new PumpFailureAlert($loc, $pumpId,
      "Pumpa " + $pumpId + " restartovana vise od 3 puta u sat vremena"));
end
```

Pravilo 4.3 - Gubitak komunikacije sa stanicom

```
rule "CEP - Gubitak komunikacije"
  when
```

```
    $loc: Location($code: locationCode)
    not(HeartbeatEvent(locationCode == $code)
        over window:time(5m))
    not ConnectionLostAlert(locationCode == $code)
  then
    insert(new ConnectionLostAlert($code,
      "Nema komunikacije sa stanicom " + $code
      + " više od 5 minuta"));
end
```

Pravilo 4.4 - Gubitak komunikacije sa pojedinačnom pumpom

```
rule "CEP - Gubitak komunikacije sa pumpom"
  when
    $ps: PumpOperationalStatus($loc: location, $pumpId: pumpId)
    not(SensorReadingEvent(sensorType == SensorType.PUMP_STATUS,
                           locationCode == $loc.code,
                           sensorName == $pumpId)
        over window:time(10m))
    not PumpConnectionLostAlert(locationCode == $loc.code, pumpId == $pumpId)
  then
    insert(new PumpConnectionLostAlert($loc.getCode(), $pumpId,
      "Nema očitavanja sa pumpe " + $pumpId
      + " na lokaciji " + $loc.getCode() + " više od 10 minuta"));
End
```

Pravilo 4.5 – Azuriranje stanja pumpe

```
rule "CEP - Azuriranje stanja pumpe"
    salience 200
    when
        $old: PumpOperationalStatus($pumpId: pumpId, $oldTs: timestamp)
        exists PumpOperationalStatus(pumpId == $pumpId, timestamp > $oldTs)
    then
        delete($old);
end
```

Pravilo 4.6 – Kvar pumpe zbog stanja FAULTY

```
rule "CEP - Kvar pumpe zbog stanja FAULTY"
    when
        $ps: PumpOperationalStatus(state == PumpState.FAULTY,
                                   $loc: location, $pumpId: pumpId)
        not PumpFailureAlert(pumpId == $pumpId)
    then
        insert(new PumpFailureAlert($loc.getCode(), $pumpId,
            "Pumpa " + $pumpId + " je u stanju FAULTY"));
end
```

Pravilo 4.7 – Obnavljanje kvara pumpe

```
rule "CEP - Obnavljanje kvara pumpe"
    when
        $alert: PumpFailureAlert($code: locationCode, $pumpId: pumpId)
        PumpOperationalStatus(state != PumpState.FAULTY,
                              pumpId == $pumpId,
                              timestamp > $alert.getTimestamp())
```

```
        then
            delete($alert);
    end
```

Pravilo 4.8 – Obnavljanje komunikacije

```
    rule "CEP - Obnavljanje komunikacije"
        when
            $alert: ConnectionLostAlert($code: locationCode)
            HeartbeatEvent(locationCode == $code,
                           this after $alert)
        then
            delete($alert);
    end
```

Pravilo 4.9 – Obnavljanje komunikacije sa pumpom

```
    rule "CEP - Obnavljanje komunikacije sa pumpom"
        when
            $alert: PumpConnectionLostAlert($code: locationCode, $pumpId: pumpId)
            SensorReadingEvent(sensorType == SensorType.PUMP_STATUS,
                               locationCode == $code,
                               sensorName == $pumpId,
                               this after $alert)
        then
            delete($alert);
    end
```

Grupa 5: Rule templates - Parametrizovani pragovi

Templejti dinamički generišu pravila za ažuriranje nekog postojećeg WaterLevelStatus-a za sva četiri nivoa vodostaja(NORMAL, ELEVATED, HIGH, CRITICAL) prema tipu lokacije.

```
template header
locationType
maxValue
statusLevel

template "water-level-below"

rule "Nivo vode - @{locationType} @{statusLevel} [@{row.rowNumber - 1}]"
    salience 100
    no-loop true
when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
                            $loc: location,
                            location.type == LocationType.@{locationType},
                            $val: value,
                            value <= @{maxValue})
    not WaterLevelStatus(location == $loc)
then
    insert(new WaterLevelStatus($loc, StatusLevel.@{statusLevel}, $val,
$reading.getTimestamp()));
end
```

```
end template


template header
locationType
minValue
maxValue
statusLevel

template "water-level-range"

rule "Nivo vode - @{locationType} @{statusLevel} [@{row.rowNumber - 1}]"
    salience 100
    no-loop true
when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
                            $loc: location,
                            location.type == LocationType.@{locationType},
                            $val: value,
                            value > @{minValue},
                            value <= @{maxValue})
    not WaterLevelStatus(location == $loc)
then
    insert(new WaterLevelStatus($loc, StatusLevel.@{statusLevel}, $val,
$reading.getTimestamp()));
end

end template


template header
locationType
maxValue
statusLevel

template "water-level-below"

rule "Nivo vode - @{locationType} @{statusLevel} [@{row.rowNumber - 1}]"
    salience 100
    no-loop true
when
    $reading: SensorReading(sensorType == SensorType.WATER_LEVEL,
                            $loc: location,
                            location.type == LocationType.@{locationType},
                            $val: value,
                            value <= @{maxValue})
    not WaterLevelStatus(location == $loc)
then
```

```
    insert(new WaterLevelStatus($loc, StatusLevel.@{statusLevel}, $val,
$reading.getTimestamp()));
end


end template
```

Tabela podataka za templejt (iste vrijednosti konceptualno odgovaraju ThresholdConfig zapisima u bazi, administrator ima dozvolu da njima upravlja – po jedan zapis po kombinaciji locationType × parameterType):

RIVER - normalMax: 200 cm, warningMax: 350 cm, criticalMax: 500 cm

CANAL - normalMax: 100 cm, warningMax: 180 cm, criticalMax: 250 cm

RESERVOIR - normalMax: 500 cm, warningMax: 700 cm, criticalMax: 850 cm

PUMP_STATION - normalMax: 150 cm, warningMax: 250 cm, criticalMax: 350 cm

Templejt generiše ukupno 16 pravila (4 nivoa klasifikacije × 4 tipa lokacije).

4. Model entiteta i činjenica sistema

Location - *Mjerna lokacija u hidrografskoj mreži*
code, displayCode, type(RIVER / CANAL / RESERVOIR / PUMP_STATION), zone, posX, posY, active, weatherCondition

Department - *Organizaciona jedinica*
code, name, description

Zone - *Geografska zona grupisanja lokacija*
code, name, description, locations

Sensor - *Senzor / mjerni instrument na lokaciji*
location, tagName, displayCode, sensorType(WATER_LEVEL / FLOW_RATE / PUMP_STATUS), unit

TagUnit - *Jedinica mjere za koju sistem koristi sledeće kodove: LEVEL_CM (cm) za nivo vode, FLOW_M3S (m³/s) za protok, PRECIP_MMH (mm/h) za trenutne padavine.*
code, unit, description

WeatherCondition - *Meteorološka stanica u bazi*
id, location, precipitation, lastUpdate

ThresholdConfig - *Konfiguracioni pragovi po kombinaciji (locationType, parameterType), jedan zapis po toj kombinaciji, upravlja isključivo administrator. Koristi ih Grupa 1 kao činjenicu u radnoj memoriji i Grupa 5 kao tabelu podataka templejta.*
id, locationType, parameterType, normalMax, warningMax, criticalMax

User - *Korisnik sistema*
id, name, lastName, email, password, role(ADMIN / OPERATOR), active, department

TrendData - *Istorijski podaci senzora*
locationCode, tagName, logTime, tagValue

WeatherObservation – *Istorijska meteorološka osmatranja*
id, locationCode, observedAt, precipitation

SensorReading - *Ulazni događaj - novo očitavanje sa senzora*

location, sensorType, sensorName, value, timestamp

WaterLevelStatus – *prvi nivo - klasifikovano stanje nivoa vode na lokaciji*

location, level: StatusLevel (NORMAL / ELEVATED / HIGH / CRITICAL), value, timestamp

FlowRateStatus – *prvi nivo - klasifikovano stanje protoka na lokaciji*

location, level: FlowLevel (LOW / NORMAL / HIGH), value, timestamp

PumpOperationalStatus – *prvi nivo - operativni status pojedinačne pumpe (jedna lokacija tipa PUMP_STATION može imati više pumpi, po jedna PumpOperationalStatus činjenica po pumpi)*

location, pumpId, state: PumpState (ACTIVE / IDLE / FAULTY / MAINTENANCE), timestamp

StationCapacity – *drugi nivo - agregirani kapacitet crpne stanice, jedna činjenica po lokaciji (dobija se agregiranjem svih PumpOperationalStatus činjenica iste lokacije)*

location: Location, level: CapacityLevel (FULL / REDUCED / MINIMAL / OFFLINE), activePumps, totalPumps

FloodRiskAssessment – *drugi nivo - procjena rizika od poplave sa obrazloženjem*

location, riskLevel: RiskLevel (LOW / MODERATE / HIGH / EXTREME), reason

InterventionRecommendation – *treći nivo - preporuka za konkretnu intervenciju*

location, type: ActionType (MONITOR / PREPARE / ACTIVATE / EVACUATE), priority, description

SystemAlert – *treći nivo - globalni nivo uzbune sistema*

level: AlertLevel (GREEN / YELLOW / ORANGE / RED), description

SensorReadingEvent - *CEP događaj - senzorsko očitavanje u stream modu, ističe nakon 2 sata*

locationCode, sensorType, sensorName, value, timestamp

HeartbeatEvent - *CEP događaj - signal živosti stanice, ističe nakon 10 minuta*

locationCode, timestamp

PumpEvent - *CEP događaj - događaj vezan za pumpu, ističe nakon 2 sata*

locationCode, pumpId, eventType: PumpEventType (START / STOP / RESTART / FAILURE), timestamp

RapidWaterLevelRise - *CEP događaj - detektovan nagli porast*

locationCode, riseCm, periodMinutes, timestamp

PumpFailureAlert - *CEP događaj - obrazac kvara pumpe*

locationCode, pumpId, description, timestamp

ConnectionLostAlert - *CEP događaj - gubitak komunikacije*

locationCode, description, timestamp

PumpConnectionLostAlert - *CEP događaj - gubitak komunikacije sa pojedinačnom pumpom*

locationCode, pumpId, description, timestamp

5. Konkretan primjer rezonovanja - korak po korak

Da bismo ilustrovali kako sistem radi, pratimo jedan konkretan dan u radu operatera. Scenario je namjerno postavljen na nivou složenosti dovoljnom da se vide sva tri nivoa rezonovanja unaprijed i jedna CEP intervencija.

Početno stanje sistema

U sistemu su registrovane dvije lokacije:

LOC_RIJEKA – tip RIVER, zona Sjever. Pragovi nivoa vode: normalMax=200cm, warningMax=350cm, criticalMax=500cm. Pragovi protoka: normalMax=2000 m³/s, warningMax=3000 m³/s, criticalMax=4000 m³/s.

LOC_PUMPA – tip PUMP_STATION, zona Jug. Pragovi nivoa vode: normalMax=150cm, warningMax=250cm, criticalMax=350cm.

Korak 1 - Prijem senzorskih podataka

U 08:00 u sesiju stiže grupa očitavanja sa obje stanice i jedan zapis o trenutnim meteorološkim uslovima. Za LOC_RIJEKA mjeri se nivo vode od 380 cm i protok od 3500 m³/s. Za LOC_PUMPA mjeri se nivo vode od 200 cm, a pet pojedinačnih očitavanja stanja pumpi pokazuje da su P1, P2 i P3 aktivne, P4 u kvaru, a P5 u stanju mirovanja. Meteorološki podaci pokazuju količinu padavina od 45 mm/h.

Korak 2 - Prvi nivo: klasifikacija stanja

Pravila Grupe 1 prevode sirova očitavanja u kategorizovana stanja. Vrijednost 380 cm na LOC_RIJEKA pada između praga upozorenja (350 cm) i kritičnog praga (500 cm), pa pravilo 1.3 daje rezultat WaterLevelStatus(LOC_RIJEKA, HIGH, 380). Protok od 3500 m³/s prelazi prag visokog protoka, pa pravilo 1.11 daje FlowRateStatus(LOC_RIJEKA, HIGH, 3500).

Na LOC_PUMPA, nivo od 200 cm je iznad normale (150 cm) ali ispod praga upozorenja (250 cm), pa pravilo 1.2 daje WaterLevelStatus(LOC_PUMPA, ELEVATED, 200). Pravila 1.15–1.18 obrađuju pet pumpi i daju pet odvojenih PumpOperationalStatus činjenica: P1, P2 i P3 sa stanjem ACTIVE, P4 sa stanjem FAULTY i P5 sa stanjem IDLE.

Korak 3 - Drugi nivo: procjena rizika i kapaciteta

Pravila Grupe 2 sada kombinuju klasifikovana stanja u procjene situacije. Za LOC_RIJEKA istovremeno postoje WaterLevelStatus(HIGH) i FlowRateStatus(HIGH). Tu situaciju pravilo 2.7 prepoznaje kao najtežu kombinaciju i daje FloodRiskAssessment(LOC_RIJEKA, EXTREME, "Visok nivo i visok protok istovremeno").

Za LOC_PUMPA, prisustvo WaterLevelStatus(ELEVATED) aktivira pravilo 2.2 i rezultat je FloodRiskAssessment(LOC_PUMPA, MODERATE, "Povišen nivo vode"). Padavine od 45 mm/h ne utiču na ovu lokaciju u ovom trenutku jer rizik već postoji u vrijednosti MODERATE.

Paralelno, pravilo 2.12 prebrojava pumpe na LOC_PUMPA preko accumulate operacije: ukupno pet pumpi, od kojih su tri u stanju ACTIVE. Odnos 3/5 = 60% spada u opseg 50–80%, što znači da stanica funkcioniše sa smanjenim kapacitetom. Rezultat je StationCapacity(LOC_PUMPA, REDUCED, 3, 5).

Korak 4 - Treći nivo: preporuke i sistemski alarm

Pravila Grupe 3 polaze od procjena rizika i pretvaraju ih u konkretne preporuke za operatera. LOC_RIJEKA ima EXTREME rizik, pa pravilo 3.5 daje InterventionRecommendation(LOC_RIJEKA, ACTIVATE, CRITICAL, "Aktivirati vanredni režim"). LOC_PUMPA ima MODERATE rizik, što pravilo 3.1 prevodi u InterventionRecommendation(LOC_PUMPA, MONITOR, MEDIUM, "Pojačan nadzor").

Globalni sistemski alarm se određuje brojem ACTIVATE preporuka u sistemu. U ovom trenutku postoji tačno jedna takva preporuka, što ispunjava uslov za narandžasti alarm. Pravilo 3.13 stoga daje SystemAlert(ORANGE, "Aktiviran vanredni režim na 1 lokaciji"). Operater na dashboardu vidi narandžasti globalni alarm.

Korak 5 - CEP: detekcija naglog porasta

U 08:25 sa LOC_RIJEKA stiže novo očitavanje sa nivoom od 510 cm. Razlika u odnosu na očitavanje od 380 cm prije 25 minuta iznosi 130 cm, što značajno prelazi prag od 50 cm u prozoru od 30 minuta. Pravilo 4.1 u CEP sesiji prepoznaje ovaj obrazac i daje izvedeni događaj RapidWaterLevelRise(LOC_RIJEKA, 130, 25), koji se odmah šalje kao notifikacija na dashboard.

Paralelno, isto očitavanje obrađuje i forward chaining sesija. Vrijednost 510 cm prelazi kritični prag od 500 cm, pa kroz pravilo 1.8 status nivoa vode za LOC_RIJEKA prelazi u WaterLevelStatus(LOC_RIJEKA, CRITICAL, 510). Procjena rizika za ovu lokaciju je već EXTREME, što je najviša vrijednost, pa eskalacija u pravilu 2.10 ne donosi nikakvu promjenu.

Korak 6 - Rezultat na dashboardu

Operater u jednom pogledu vidi globalni narandžasti alarm, LOC_RIJEKA u stanju vanrednog režima sa preporukom ACTIVATE i CEP notifikacijom o naglom porastu od 130 cm u zadnjih 25 minuta, LOC_PUMPA u stanju pojačanog nadzora sa MODERATE rizikom i sa naznakom da kapacitet stanice radi na 60% (tri od pet pumpi). Umjesto da samostalno povezuje očitavanja sa pet pumpi, dvije lokacije, meteorološku sliku i temporalni obrazac porasta, operater dobija sintetizovanu procjenu sa konkretnim preporukama za intervenciju.